

AIAC-Ass-14.4

Name:T.Shivani

Ht.No:2303A51312

Batch:05

Prompt: Create a responsive startup homepage using HTML and CSS.

The webpage should include:

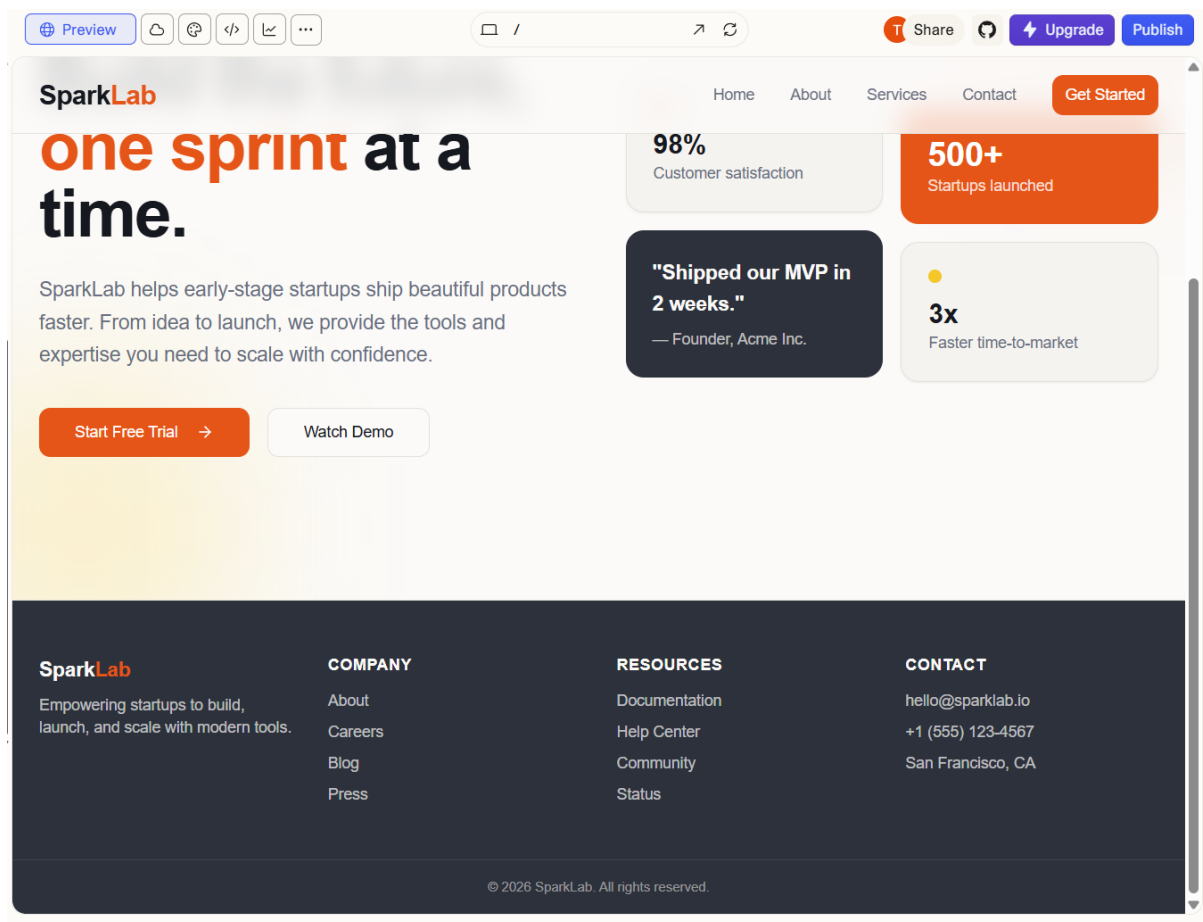
- A header section with a logo and navigation links (Home, About, Services, Contact).
- A main content section with a heading, short description, and a call-to-action button.
- A footer section with basic company information and copyright text.

Use CSS media queries to make the layout responsive for mobile, tablet, and desktop screens.

Requirements:

- Use clean and well-structured HTML.
- Use Flexbox or Grid for layout.
- Add comments explaining important parts of the code.
- Ensure the layout adjusts smoothly for different screen sizes.

Output:



Explanation: his task creates a **responsive homepage** for a startup using **HTML and CSS**.

- **HTML** is used to build the structure of the webpage. It includes three main parts:
 - **Header** – contains the logo and navigation links (Home, About, Services, Contact).
 - **Main Content** – displays the main information about the startup with a heading, description, and button.
 - **Footer** – shows basic company information and copyright text.
- **CSS** is used to style the webpage and make it responsive.
- **Media queries** are used in CSS to adjust the layout for different screen sizes like **mobile, tablet, and desktop**.

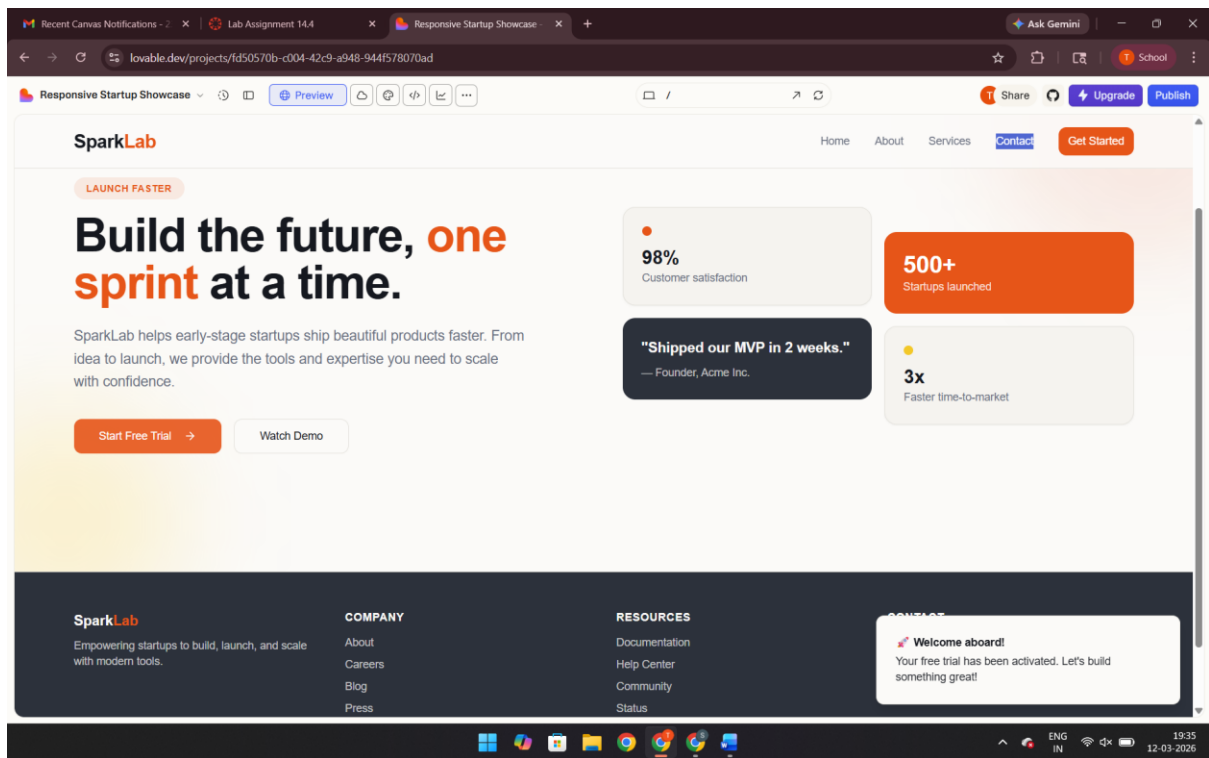
As a result, the webpage automatically **changes its layout and looks good on all devices** while keeping the code clean and organized.

Prompt: Create a simple web page using **HTML and JavaScript** that includes a **Call-to-Action button**.

When the user clicks the button, **display a message or alert** without reloading the page.

Write **clean and readable JavaScript code** and add **meaningful comments explaining the logic** of the script.

Output:



Explanation: In this task, a Call-to-Action (CTA) button is added to a webpage using HTML.

JavaScript is used to make the button interactive. When the user clicks the button, a function runs that displays a message or alert on the screen.

Comments are included in the JavaScript code to explain how the click event works and how the message is shown.

The result is a webpage where clicking the button gives an immediate response without refreshing the page.

Prompt: Create a contact form using HTML and JavaScript with fields for Name, Email, and Message.

Add client-side validation using JavaScript that prevents empty form submission and checks whether the email is in a valid format.

Show inline error messages if the input is invalid and display a success message when the form is filled correctly.

Include comments in the JavaScript code to explain the validation logic.

Output:

The image displays two screenshots of a web application interface, likely a contact form, showing the state before and after a message is sent.

Top Screenshot (Before Submission):

- Header:** SparkLab logo, navigation links (Home, About, Services, Contact), and a "Get Started" button.
- Section:** "Get in Touch" with the subtext "Have a question or want to work together? Drop us a message."
- Form Fields:**
 - Name:** Input field with placeholder text "Your name".
 - Email:** Input field with placeholder text "you@example.com".
 - Message:** Textarea with placeholder text "Your message...".
- Action:** A large orange "Send Message" button with a paper plane icon.

Bottom Screenshot (After Submission):

- Header:** Same as the top screenshot.
- Section:** "Get in Touch" with the same subtext.
- Form Fields:** The input fields are now empty, indicating successful submission.
- Action:** The "Send Message" button remains visible.
- Footer:** SparkLab logo, tagline "Empowering startups to build, launch, and scale with modern tools.", and a "COMPAN" link.
- Notification:** A green checkmark icon and the text "Message sent! Thanks for reaching out. We'll get back to you soon." are displayed in a white box at the bottom right.

Explanation: This task creates a contact form on a website to collect user queries.

- HTML is used to design the form with Name, Email, and Message fields.
- JavaScript is used for client-side validation.
- It checks if the fields are empty and verifies the email format before submission.
- If the input is wrong, error messages appear near the fields.
- If all inputs are correct, a success message is displayed.

This validation happens on the client side, meaning it runs in the user's browser without sending incorrect data to the server.

Prompt: Create a simple **product list webpage using HTML and JavaScript**.

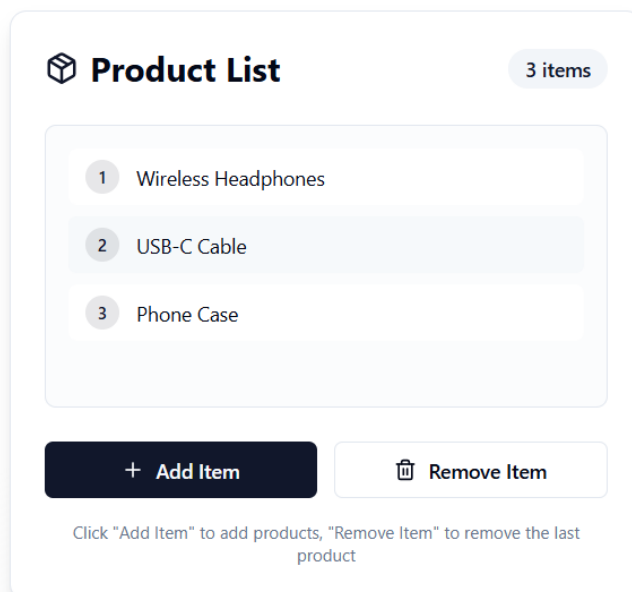
The page should contain a **list of items** and two buttons: **Add Item** and **Remove Item**.

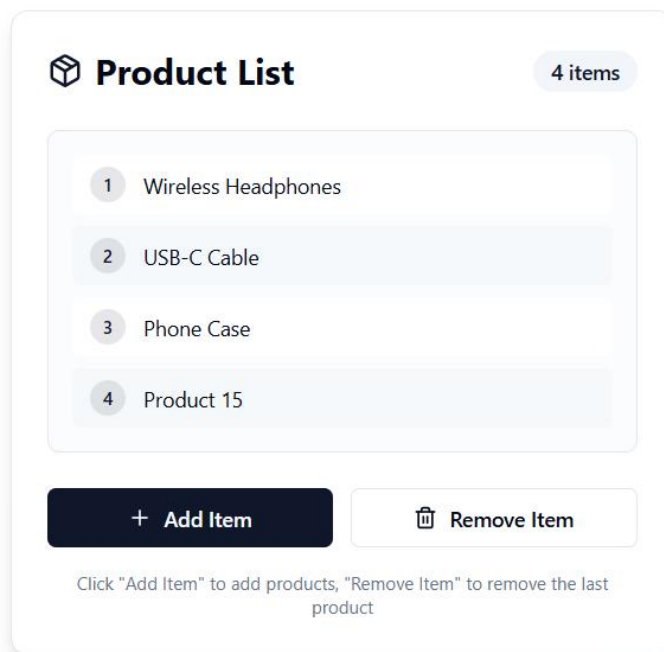
Write JavaScript code that:

- **Adds a new item** to the list when the **Add button** is clicked.
- **Removes the last item** from the list when the **Remove button** is clicked.
- Updates the **DOM dynamically without reloading the page**.

Ensure the JavaScript code is **clean, readable, and includes comments explaining the logic**.

Output:





Explanation: In this task, a **dynamic product list** is created using **HTML and JavaScript**.

- **HTML** is used to create the list and buttons (Add and Remove).
- **JavaScript** is used to modify the **DOM (Document Object Model)**.

When the **Add button** is clicked, JavaScript **creates and adds a new list item** to the page.

When the **Remove button** is clicked, JavaScript **removes the last item from the list**.

All updates happen **instantly without refreshing the page**, making the webpage **interactive and dynamic**.

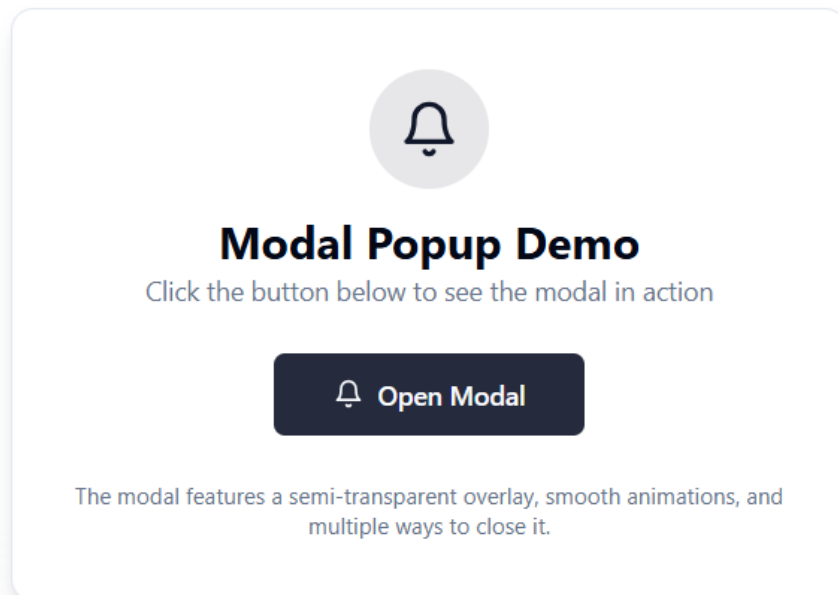
Prompt: Create a **modal popup notification webpage** using **HTML, CSS, and JavaScript**. The page should include a button called **"Open Modal"**.

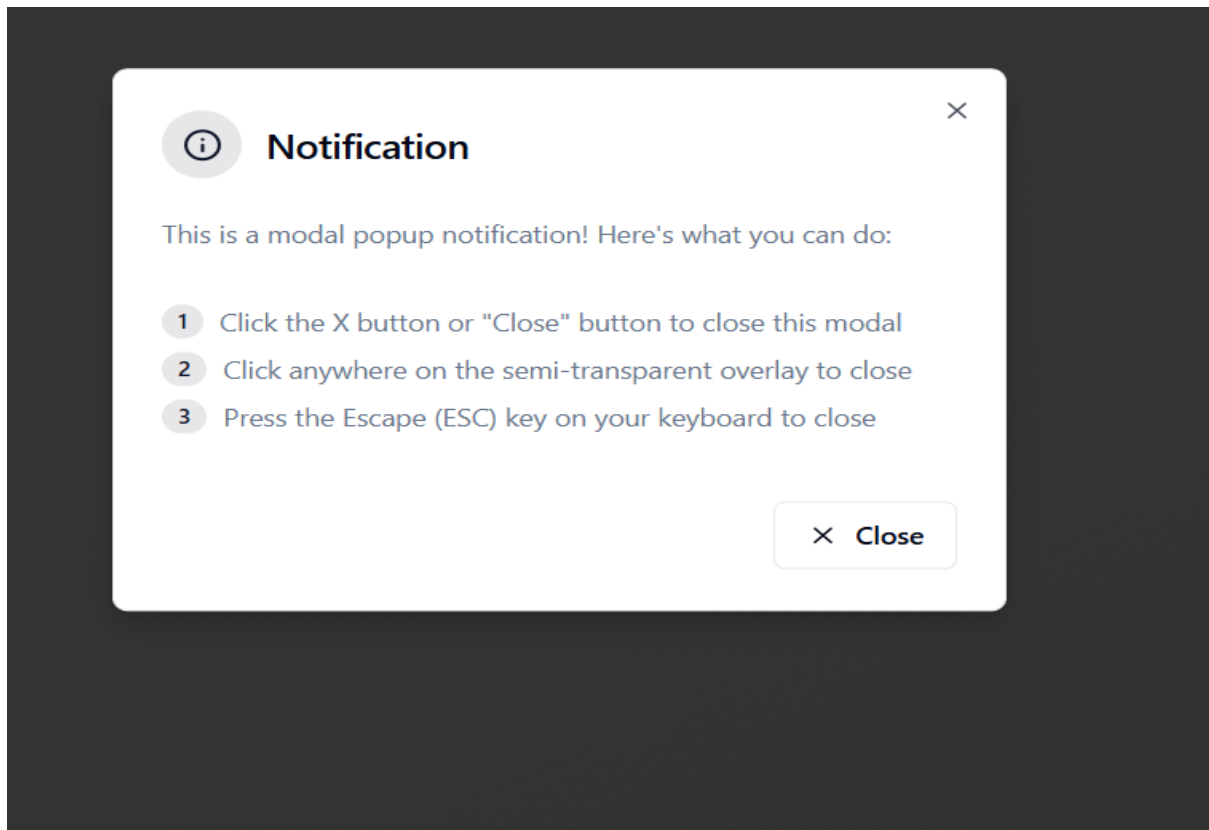
Requirements:

- Write **HTML structure** for the modal popup.
- Use **CSS** to style the modal with a **semi-transparent background overlay**.
- Add **JavaScript** to **open and close the modal**.
- The modal should close when the user clicks the **close button** or the **overlay area**.

- Add **comments in the code** explaining how the modal works.

Output:





Explanation: This task creates a **modal popup notification** for a website.

- **HTML** is used to create the **modal structure and Open Modal button**.
- **CSS** is used to style the popup and add a **semi-transparent background overlay** so the modal appears above the page content.
- **JavaScript** controls the **opening and closing of the modal**.

When the **Open Modal button** is clicked, the popup appears on the screen.

The modal can be closed by clicking the **close button or the overlay area**.

This creates a **clear notification popup without reloading the page**.